

Assignment 6

1. Assignment Description

Problem: Vector Shape Simplification (50 points)

In Assignment 3, you implemented an L-System to procedurally generate complex structures. You may have noticed that when applying string rewriting rules over a high number of iterations, the system generates a massive number of line segments. Attempting to render these unoptimized shapes directly on the screen using a graphical user interface (GUI) or writing them to an image file like an `.svg` can cause severe performance lag and unnecessarily large file sizes.

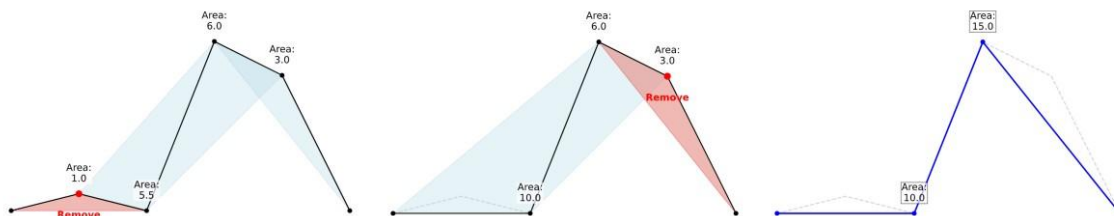
To solve this, you will implement the Visvalingam-Whyatt algorithm to simplify 2D vector polylines. This algorithm progressively removes the "least important" vertices from a shape, preserving its overall visual characteristics while drastically reducing the number of points.

Visvalingam-Whyatt Algorithm

The core concept is to evaluate the "importance" of a vertex based on the area of the triangle it forms with its two immediate neighbors.

- **Initialization:** Load all the initial points into a List. The first and last points are endpoints and must never be removed.
- **Area Calculation:** For every intermediate point (current), calculate the area of the triangle formed by its previous node, the current node, and its next node. The formula for the area of a triangle given three points A, B, and C is: $\text{Area} = 0.5 * \text{abs}(x_A * (y_B - y_C) + x_B * (y_C - y_A) + x_C * (y_A - y_B))$.
- **Find the Minimum:** Scan the list to find the vertex with the smallest calculated area. This vertex contributes the least to the overall shape.
- **Node Removal:** Delete this minimum-area vertex from your Doubly Linked List.
- **Recalculation:** After removing the vertex, its `prev` and `next` neighbors are now connected to each other. You must recalculate the triangle areas for these two newly adjacent neighbors.
- **Termination:** Repeat the finding, removing, and recalculating steps until the total number of remaining points matches the target point count.

Example



- **Step 0: Input Vertices and Target Number.** Given an initial polyline of 6 points: $(0, 0), (2, 0.5), (4, 0), (6, 5), (8, 4), (10, 0)$. Our target is to simplify it to 4 points. The first and last points are endpoints and are never removed.
- **Step 1: Initial Area Calculation.** Calculate the "effective area" for all inner vertices using the triangle formed by the vertex and its immediate left and right neighbors.

Here, point $(2, 0.5)$ forms the smallest triangle area (1.0) . It is the "least important" point.

- **Step 2: Vertex Removal.** Remove the point $(2, 0.5)$. Its neighbors $(0, 0)$ and $(4, 0)$ now connect directly. Because the neighbors have changed, we *must* recalculate the area for $(4, 0)$ using its new left neighbor $(0, 0)$. (Its area increases from 5.5 to 10.0).
- **Step 3: Vertex Removal.** Find the new minimum area among the remaining inner points. Now, point $(8, 4)$ has the smallest area (3.0) . Remove it, connect its neighbors $(6, 5)$ and $(10, 0)$, and recalculate their areas.
- **Step 4: Final Result.** The algorithm terminates because we have reached the target of 4 points: $(0, 0)$, $(4, 0)$, $(6, 5)$, $(10, 0)$. The major visual feature (the peak at $(6, 5)$) is successfully preserved while minor noise is discarded.

Hints

To achieve high performance on massive datasets, your implementation needs to utilize a combination of specific data structures to optimize the time complexity $O(N \log N)$:

- **Linked List for $O(1)$ Deletions:** Because this algorithm requires frequent and dynamic deletions of arbitrary points from the middle of a large dataset, you must utilize a doubly linked list. This ensures efficient node removal, completely avoiding the massive $O(N)$ performance penalty (and resulting execution timeouts) of shifting or copying an entire array every time a single vertex is deleted.
- **Min-Heap for $O(\log N)$ Searches and Updates:** The core of the algorithm requires repeatedly finding the vertex with the absolute smallest triangle area. Linearly scanning a linked list to find this minimum every time would result in an extremely slow $O(N^2)$ time complexity. To optimize this, you must maintain a Min-Heap (Priority Queue). The heap allows you to extract the minimum-area vertex in $O(\log N)$ time. Furthermore, when a vertex is removed, the areas of its adjacent neighbors change; the heap allows you to efficiently update and re-sort these modified areas in $O(\log N)$ time.

Input

- The first line contains an integer T , representing the number of test cases. Each test case consists of multiple lines:
 1. M and N : Two integers separated by a space. N represents the initial number of points, and M represents the target number of points after simplification.
 2. **Points:** The next P lines each contain two floating-point numbers, representing the x and y coordinates of a vertex sequentially.
- Constraints
 - number of test cases T : $1 \leq T \leq 20$.
 - Initial points N : $3 \leq N \leq 100,000$.
 - Target points M : $2 \leq M < N$.

Output

- For each test case, output the final remaining T points sequentially to the terminal.
Each line must contain exactly 1 point representing a vertex in the format: x, y . You must round coordinates to 2 decimal places for better readability.

Sample Input

```
1
3 5
0.0 0.0
5.0 1.0
10.0 0.0
5.0 10.0
0.0 0.0
```

Sample Output

```
0.00,0.00
5.00,10.00
0.00,0.00
```

2. Submission Guidelines

Please organize your code and files according to the following directory structure and compress them into a single `.zip` archive before uploading.

- Single Submission Archive (`.zip`): Place your main function code (`main.cpp`), all custom `.cpp` and `.h` files, and any other files required for compilation into a specific root folder. Compress this folder into a `.zip` file. (Note: If you organize your source codes into multiple subdirectories, you must include a `Makefile` in the root of your folder.)
- The `build` Directory: You must create a subfolder named `build` inside your root directory. This folder must contain your compiled executable file (`.exe`).

Any required dependencies (e.g., `.dll` files) should also be placed in this folder.

Naming Example & Directory Structure:

- Zip File Name: `1142DS_<Student ID>_a6.zip`
(e.g., `1142DS_B112345678_a6.zip`)

Your `.zip` archive must follow this file structure:

```
1142DS_B123456789_a6.zip/
├─build/ # Must contain executable
│   └─1142DS_B123456789_a6.exe
│       └─... # dependencies(.dll)
└─main.cpp
```

```
└...    # And any other .cpp/.h/Makefile files
```

3. Grading Policies

- Coding Style (10 points): Assessed based on code readability, consistent indentation, and meaningful naming conventions. Arbitrary or meaningless variable names (e.g., a, b, c, aa, bb, cc) are strictly prohibited and will result in point deductions.
- Comments (5 points): Assessed based on brief and clear explanations of your function purposes and the overall processing flow. Focus on explaining the core logic. Do not add trivial or redundant comments for basic C++ syntax (e.g., commenting on `#include` or `using namespace std`). Additionally, every source file must include a file header specifying the author (including student ID), the date of writing (e.g., Mar. 22, 2026), and a description of the program's functionality and usage.
- Correctness (35 points): Points are awarded based on the percentage of test cases your program successfully passes.

4. Restrictions (Not Allowed)

You are permitted to use C++ Standard Template Library (STL) containers that correspond to data structures you have already implemented or learned in previous assignments, such as `<vector>`, `<list>`, and `<set>`. However, the usage of STL containers, algorithms, or libraries related to the current assignment or upcoming lectures—specifically `std::priority_queue`, `std::map`, `std::sort`, or similar advanced structures—is strictly prohibited. You must implement the core data structures for this assignment (e.g., the Min-Heap) from scratch. If any prohibited libraries or algorithms are used, the correctness score will be automatically marked as 0.